

초보자 기준 인프라 알아보기

☉ 선택

Infra

[인프라](#)

[전체 구조부터 먼저 보기](#)

[1단계: EC2 접속하기](#)

[EC2란?](#)

[SSH 접속이란?](#)

[의미](#)

[pem 키 권한 400은 왜 해?](#)

[2단계: 방화벽\(UFW\) 설정하기](#)

[UFW란?](#)

[포트란?](#)

[왜 22번부터 열어야 해?](#)

[기본 흐름](#)

[왜 SSH 터미널을 2~3개 켜두라고 할까?](#)

[3단계: Docker 설치하기](#)

[Docker란?](#)

[Docker 설치 후 하는 일](#)

[4단계: Docker Compose로 여러 서비스 한 번에 관리하기](#)

[Docker Compose란?](#)

[실행 명령어](#)

[종료 명령어](#)

[docker-compose.yml 읽는 법](#)

`"5000:8080"` ?

[volume?](#)

[network?](#)

[5단계: Jenkins로 자동 배포하기](#)

[Jenkins란?](#)

[Jenkinsfile이란?](#)

[withCredentials는 왜 써?](#)

[백엔드 배포 흐름 이해하기](#)

[Dockerfile\(백엔드\)은 뭘까?](#)

[프론트 배포 흐름 이해하기](#)

[프론트는 왜 백엔드랑 달라?](#)

[프론트 배포 흐름](#)

[프론트 Dockerfile이 2단계인 이유](#)

[의미](#)

[장점](#)

6단계: Nginx 설정하기

Nginx란?

역할 1. 프론트 파일 보여주기

역할 2. API 요청 전달하기

예시 설정

7단계: HTTPS 적용하기

HTTPS란?

certbot?

자동 갱신은 왜 해?

8단계: Portainer로 도커 쉽게 관리하기

Portainer란?

9단계: ELK로 로그 보기

ELK란?

왜 Elasticsearch에서 서버가 터질 수 있어?

자주 쓰는 용어 정리

초보자 기준으로 실제 구축 순서 정리

1) EC2 만들기

2) SSH 접속하기

3) 방화벽 열기

4) Docker 설치

5) docker-compose.yml 작성

6) 컨테이너 띄우기

7) Jenkins 접속

8) GitLab 웹훅 연결

9) Jenkinsfile 작성

10) Nginx 설치 및 설정

11) HTTPS 적용

12) 운영 보조 도구 붙이기

초보자가 특히 조심할 점

인프라

인프라 = 우리가 만든 서비스를 실제 서버 위에서 돌아가게 만드는 환경

전체 구조부터 먼저 보기

사용자

↓

도메인 접속

↓

Nginx

└─ 프론트 파일 보여줌

└─ /api 요청은 백엔드로 전달

↓

Spring Boot

↓

MySQL / Redis / Kafka

그리고 배포 자동화:

GitLab → Jenkins → Docker 배포

1단계: EC2 접속하기

EC2란?

AWS에서 빌려 쓰는 가상 컴퓨터

SSH 접속이란?

```
ssh -i 키파일.pem ubuntu@서버주소
```

이건 내 컴퓨터에서 EC2 서버로 들어가는 명령어

의미

- `ssh` : 원격 접속
- `i 키파일.pem` : 접속용 열쇠 사용
- `ubuntu` : 서버 계정 이름
- `서버주소` : EC2 IP 주소

pem 키 권한 400은 왜 해?

```
chmod400 키파일.pem
```

이건 키 파일을 안전하게 보호하기 위한 설정

2단계: 방화벽(UFW) 설정하기

UFW란?

우분투의 방화벽

포트란?

포트는 서버 안의 방 번호

예를 들어:

- 22 = SSH 방
 - 80 = 웹 접속 방
 - 443 = HTTPS 방
 - 8080 = 백엔드 방
 - 3306 = MySQL 방
-

왜 22번부터 열어야 해?

```
sudo ufw allow22
```

22번은 SSH 접속용 포트

기본 흐름

```
# 현재 방화벽 상태 확인
sudo ufw status
# 필요한 포트 열기
sudo ufw allow22
sudo ufw allow80
sudo ufw allow443
# 방화벽 켜기
sudo ufw enable
```

```
# 열린 포트 확인하기
sudo ufw status numbered
```

왜 SSH 터미널을 2~3개 켜두라고 할까?

방화벽 설정을 잘못하면 현재 접속이 끊기고 다시 못 들어갈 수 있어서 여러 터미널을 켜두면 하나가 끊겨도 다른 창에서 복구할 가능성이 생김

3단계: Docker 설치하기

Docker란?

프로그램을 컨테이너라는 독립된 상자 안에 넣어서 실행하는 기술

Docker 설치 후 하는 일

```
# 도커 설치
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io
# 도커 실행
sudo systemctl start docker
# 서버 재부팅돼도 자동 실행
sudo systemctl enable docker
# 버전 확인
docker --version
```

4단계: Docker Compose로 여러 서비스 한 번에 관리하기

Docker Compose란?

여러 개의 컨테이너를 한 번에 정의하고 실행하는 도구

- Jenkins
- MySQL

- Redis
- Zookeeper
- Kafka
- Portainer

`docker-compose.yml` 파일에 "이 서비스들을 이렇게 실행해줘"라고 적어두는 것

실행 명령어

```
# 컨테이너 올려서 백그라운드 실행하기
docker compose up -d
```

종료 명령어

```
docker compose down
```

docker-compose.yml 읽는 법

```
services:
  jenkins:
    container_name: jenkins
    ports:
      - "5000:8080"
```

- `jenkins` 라는 서비스를 만들고
- 컨테이너 이름은 `jenkins`
- 서버의 5000번 포트와
- 컨테이너 안의 8080번 포트를 연결

"5000:8080" ?

서버포트 : 컨테이너포트

- 외부에서는 5000으로 접속
 - 실제 컨테이너 안에서는 8080으로 동작
-

volume?

```
volumes:  
  - jenkins_home:/var/jenkins_home
```

데이터를 영구 저장하기 위한 공간 연결

network?

```
networks:  
  - jenkins-network
```

컨테이너끼리 같은 내부 네트워크에서 통신하게 해주는 것

5단계: Jenkins로 자동 배포하기

Jenkins란?

코드가 바뀌면 자동으로 빌드하고 배포해주는 도구

Jenkinsfile이란?

Jenkins에게 "배포를 어떤 순서로 할지" 적어둔 작업 지시서

예를 들어 백엔드 Jenkinsfile 흐름은:

1. (.env) 꺼내기
 2. Gradle 빌드
 3. Docker 이미지 생성
 4. 기존 컨테이너 중지
 5. 새 컨테이너 실행
-

withCredentials는 왜 써?

```
withCredentials([
    file(credentialsId: 'env-file', variable: 'EnvFile'),
]) {
```

`.env` 파일에는 중요한 것들이 많으니 Jenkins의 Credentials에 따로 저장해두고, 배포할 때만 잠깐 꺼내서 사용하는 것

백엔드 배포 흐름 이해하기

```
GitLab push
→ Jenkins 웹훅 실행
→ 코드 가져오기
→ Gradle build
→ jar 생성
→ Docker 이미지 생성
→ 기존 backend 종료
→ 새 backend 실행
```

즉, 개발자가 코드를 push하면 Jenkins가 알아서 새 버전 서버로 교체해주는 구조

Dockerfile(백엔드)은 뭘까?

```
# Java 17 환경 준비
FROM bellsoft/liberica-openjdk-alpine:17
# 작업 폴더 /app
WORKDIR /app
# 빌드된 jar 넣기
COPY build/libs/xxx.jar app.jar
# 8080 포트 사용
EXPOSE 8080
# 컨테이너 시작 시 자바 실행
ENTRYPOINT ["java", "-jar", "app.jar"]
```

스프링부트 실행용 컨테이너를 만드는 설명서

프론트 배포 흐름 이해하기

프론트는 왜 백엔드랑 달라?

배포할 때는 보통 정적 파일로 만들어지기 때문

프론트 배포 흐름

```
GitLab push
→ Jenkins 실행
→ npm install
→ npm run build
→ dist 폴더 생성
→ Nginx가 이 dist 파일들을 사용자에게 제공
```

즉, 프론트는 실행 중인 서버라기보다 **완성된 웹 파일 세트**

프론트 Dockerfile이 2단계인 이유

```
FROM node:18AS build
...
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

의미

1. Node 환경에서 프론트 빌드
2. 완성된 파일만 Nginx로 넘김

장점

- 최종 이미지가 가벼움
 - 빌드 도구가 안 남음
 - 배포가 깔끔함
-

6단계: Nginx 설정하기

Nginx란?

Nginx는 웹 서버이자 리버스 프록시

역할 1. 프론트 파일 보여주기

사용자가 사이트에 접속하면 HTML/CSS/JS 파일을 전달해줌

역할 2. API 요청 전달하기

사용자가 `/api` 요청을 보내면 백엔드(Spring Boot)에게 넘겨줌

예시 설정

```
server {  
    listen 80;  
    root /home/ubuntu/nginx;  
    index index.html;  
  
    location / {  
        try_files $uri /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://localhost:8080;  
    }  
}
```

`location /`

프론트 페이지 요청 처리

`location /api/`

API 요청은 Nginx가 직접 처리하지 않고 8080에서 도는 백엔드로 넘김

7단계: HTTPS 적용하기

HTTPS란?

암호화된 웹 통신

certbot?

무료 SSL 인증서를 발급받는 도구

자동 갱신은 왜 해?

인증서는 기간이 있어서 만료돼서 certbot renew 같은 걸 주기적으로 돌려줘야 함

`renew-cert.sh` 와 cron 설정은 그걸 자동화하는 작업

8단계: Portainer로 도커 쉽게 관리하기

Portainer란?

도커 컨테이너를 웹 화면으로 쉽게 관리하는 도구 (관리자 화면)

9단계: ELK로 로그 보기

ELK란?

로그를 모아서 보기 쉽게 만드는 도구 세트

- **Elasticsearch** = 저장/검색
 - **Logstash** = 수집/가공
 - **Kibana** = 화면에서 보기
-

왜 Elasticsearch에서 서버가 터질 수 있어?

Elasticsearch는 메모리를 많이 먹어.

그래서 작은 서버에서 그냥 띄우면 서버가 버벅이거나 죽을 수 있어.

```
ES_JAVA_OPTS=-Xms1g -Xmx1g
```

이건 엘라스틱이 메모리를 너무 많이 못 먹게 제한하는 거야.

자주 쓰는 용어 정리



서버

서비스가 돌아가는 컴퓨터

배포

내 코드를 실제 서버에 올리는 것

포트

서버 안에서 서비스 구분용 번호

컨테이너

앱을 실행하는 독립된 상자

이미지

컨테이너를 만들기 위한 설계도

볼륨

컨테이너가 지워져도 남는 저장 공간

웹훅

GitLab이 Jenkins에게 “코드 바뀌었어”라고 알려주는 신호

리버스 프록시

Nginx가 앞에서 요청 받고 뒤 서버로 전달하는 방식

정적 파일

HTML/CSS/JS처럼 그대로 제공되는 파일

CI/CD

- CI: 자동 빌드/테스트
- CD: 자동 배포

초보자 기준으로 실제 구축 순서 정리

1) EC2 만들기

- AWS에서 EC2 생성
- pem 키 받기
- 공인 IP 확인

2) SSH 접속하기

```
chmod400 키파일.pem  
ssh -i 키파일.pem ubuntu@서버주소
```

3) 방화벽 열기

```
sudo ufw allow22  
sudo ufw allow80  
sudo ufw allow443  
sudo ufw allow5000  
sudo ufw allow9000  
sudo ufw enable
```

4) Docker 설치

- 도커 설치
 - 자동 실행 설정
 - 버전 확인
-

5) docker-compose.yml 작성

- Jenkins
- MySQL
- Redis
- Kafka
- Portainer

같은 서비스 정의

6) 컨테이너 띄우기

```
docker compose up-d  
dockerps
```

7) Jenkins 접속

- 서버IP:5000
 - 초기 비밀번호 확인
 - 플러그인 설치
-

8) GitLab 웹훅 연결

- GitLab Access Token 생성
 - Jenkins와 GitLab 연결
 - push 시 자동 배포되게 설정
-

9) Jenkinsfile 작성

- 백엔드 빌드 및 docker run
 - 프론트 빌드 및 dist 생성
-

10) Nginx 설치 및 설정

- 프론트 정적 파일 제공
 - `/api` 는 백엔드로 프록시
-

11) HTTPS 적용

- certbot 설치
 - 인증서 발급
 - 자동 갱신 설정
-

12) 운영 보조 도구 붙이기

- Portainer
 - ELK
-

초보자가 특히 조심할 점

1. SSH 포트 안 열고 UFW 켜면 접속 끊길 수 있음
2. 볼륨 안 쓰면 Jenkins 데이터 날아갈 수 있음
3. 프론트와 백엔드는 배포 방식이 다름
 - 백엔드 = 실행 중인 서버
 - 프론트 = 정적 파일 배포
4. DB나 Redis 포트는 외부에 꼭 열 필요가 없을 수도 있음
보안상 내부 통신만 허용하는 게 더 좋을 때가 많아.
5. Kafka 설정은 localhost 때문에 꼬일 수 있음
6. Redis 기본 포트는 보통 6379